



DAY 20: CALLBACK FUNCTIONS IN JAVASCRIPT 📞💡



◆ What is a Callback Function?

A callback function is a function that is passed as an argument to another function and is executed later, after the completion of that function.

✓ Simple Example of a Callback Function

```
function greet(name, callback) {  
    console.log(`Hello, ${name}!`);  
    callback(); // Calling the callback function  
}  
function sayGoodbye() {  
    console.log("Goodbye! Have a great day!  
👋");  
}  
// Passing sayGoodbye as a callback to greet  
function  
greet("John", sayGoodbye);
```

Output:

Hello, John!

Goodbye! Have a great day! 🖐️

👉 Here, sayGoodbye is a callback function that runs after the greet function completes execution.

◆ Why Do We Need Callback Functions?

1. Handling Asynchronous Operations (APIs, Timers, File Handling).
2. Ensuring Code Executes in the Right Order (Avoid race conditions).
3. Improving Code Reusability (Functions can be dynamic and reusable).

◆ Callback Function in setTimeout()

JavaScript's `setTimeout()` function uses callbacks to execute code after a delay.

```
console.log("Start");  
setTimeout(() => {  
    console.log("This message appears  
after 2 seconds!");  
}, 2000);  
console.log("End");
```



Output:

Start

End

(This message appears after 2 seconds!)

👉 `setTimeout()` waits 2 seconds before executing the callback function.

◆ Callbacks in Asynchronous Operations (Example: Fetching Data)

In real-world applications, we often use callbacks to handle API requests.

```
function fetchData(callback) {  
  console.log("Fetching data... ⌚");  
  setTimeout(() => {  
    const data = { name: "Alice", age: 25 };  
    callback(data); // Calling the callback  
function after fetching data  
  }, 2000);  
}  
function displayData(data) {  
  console.log("Data received:", data);  
}  
// Calling fetchData and passing displayData as a  
callback  
fetchData(displayData);
```

Output:

```
Fetching data...   
(Data received: { name: "Alice", age: 25 })
```

👉 The `displayData` function is executed only after data fetching is complete.

◆ Callback Hell (The Downside of Callbacks)

When multiple callbacks are nested, it leads to messy and unreadable code, known as "callback hell."

Example: Multiple Nested Callbacks (Callback Hell 🤔)

```
function step1(callback) {  
    setTimeout(() => {  
        console.log("Step 1  
complete");  
        callback();  
    }, 1000);  
}
```

```
function step2(callback) {  
    setTimeout(() => {  
        console.log("Step 2  
complete");  
        callback();  
    }, 1000);  
}
```

```
function step3(callback) {  
  setTimeout(() => {  
    console.log("Step 3 complete");  
    callback();  
  }, 1000);  
}
```

```
// Callback hell starts here ↓  
step1(() => {  
  step2(() => {  
    step3(() => {  
      console.log("All steps completed!");  
    });  
  });  
});
```

Output:

```
Step 1 complete  
Step 2 complete  
Step 3 complete  
All steps completed!
```

◆ How to Avoid Callback Hell?

- Use Promises – Improves readability (.then()) instead of nested callbacks).
- Use Async/Await – More structured and readable async code.